IAO Belgium — Frontend Code Audit

Audited app: `IAO-Belgium-Frontend` (React 19 + Vite 7 SPA)
Stack: React 19, Vite 7, TanStack Router + TanStack Query, Zustand, Axios, Tailwind 4, react-hook-form + Zod, i18next
Audit date: 2026-06-24
Branch: `test` (UAT)
Scope: `src` (~40,800 LOC, 271 JS/JSX files). **Frontend only** — the backend has its own separate audit.
Environment audited: UAT (Belgium / EU-zone).

> **How to read this report.** Findings are tagged with a severity:
> 🔴 **Critical**, 🟠 **High**, 🟡 **Medium**, 🔵 **Low / hygiene**.
> Each lists evidence (`file:line`), impact, and a concrete fix.
> A prioritized remediation list is at the end.

> ⚠️ **A frontend cannot enforce security.** Everything in a SPA ships to the user's browser and can be read, modified, and bypassed. Route guards, permission checks, and hidden menu items here are **UX**, not access control. The real boundary is the API. Several findings below are only *real* problems because the backend does not yet enforce the same rules. Where that's the case, it is called out explicitly.

---

0. Executive Summary

The frontend is **well-structured and modern**: clean `api / store / pages / components / router` separation, a sensible Axios interceptor with silent token-refresh, in-memory access token (not `localStorage` — good), HttpOnly refresh cookie, i18n, and a permission-aware admin sidebar. The team clearly knows the stack.

The audit found **no catastrophic client-only defect**, but several issues that matter — most importantly **unsanitized HTML rendering (XSS)**, an **asymmetric route-guard model** (admin routes are permission-gated, teacher routes are not), and the fact that **all client-side "protection" is cosmetic until the backend enforces RBAC**.

Top things to fix

#	Finding	Severity	Why it matters
1	**Unsanitized `dangerouslySetInnerHTML`** in 6 places (notifications)	🔴 Critical	Stored XSS — attacker-authored notification HTML executes in admin/teacher browsers.
2	**Client-side guards are the *only* access control**	🔴 Critical	A user can edit JS / call the API directly and bypass every guard. Real fix is server-side (backend RBAC).
3	**Teacher routes have no permission guard**	🟠 High	`adminRoutes` wrap every page in `ProtectedRoute`; `teacherRoutes` wrap nothing. Inconsistent + relies entirely on role string.
4	**API key shipped to the browser** (`VITE_APP_API_KEY`)	🟠 High	Any Vite `VITE_*` var is embedded in the bundle and world-readable. It is not a secret.
5	**Hardcoded cross-app URLs** (`student-iao.xyvin.com`, unpkg CDN)	🟡 Medium	Environment-coupling + supply-chain/CDN dependency for the PDF worker.

---

1. Authentication & Session Handling

1.1 🔵 GOOD — Access token kept in memory, refresh token in HttpOnly cookie

Evidence: `src/store/useAuthStore.js` holds `token` in Zustand state only (never persisted to `localStorage`). `src/api/axiosintercepter.js:9` sets `withCredentials: true`; the refresh token lives in the backend's HttpOnly cookie.
Assessment: This is the correct pattern and resists XSS token theft. Keep it. **Note** this strength is undermined by Finding 3.1 (XSS) — if XSS exists, an attacker doesn't need the token; they act as the user in-session.

1.2 🟡 MEDIUM — Silent refresh has no single-flight guard (refresh stampede)

Evidence: `src/api/axiosintercepter.js:33-52`. On a 401, each failed request independently calls `refreshAccessToken()`. If several requests 401 at once (common on token expiry with a dashboard firing many queries), they all trigger parallel `/auth/refresh` calls.
Impact: Race conditions; with backend refresh-token rotation this can revoke the just-issued token and log the user out spuriously.
Fix: Add a module-level `refreshPromise` so concurrent 401s await one shared refresh:

```
let refreshPromise = null;
if (!refreshPromise) refreshPromise = useAuthStore.getState().refreshAccessToken().finally(() => {
refreshPromise = null; });
await refreshPromise;
```

1.3 🟡 MEDIUM — `initializeAuth` blocks first paint with no failure UX

Evidence: `src/main.jsx:17` calls `initializeAuth()` at module load; `useAuthStore.js:137-149` calls `/auth/refresh` immediately on every cold load — including for never-logged-in visitors, who get a guaranteed 401. Functionally fine, but it's an avoidable failed request on every first visit and couples app boot to network latency.
Fix: Skip the refresh attempt when there's no prior session hint, or surface a fast unauthenticated path.

1.4 🟡 MEDIUM — No global handling of `403`

Evidence: The interceptor handles `401` (refresh) but not `403`. Once the backend adds RBAC, teachers hitting admin APIs will get `403`, which currently surfaces as an unhandled rejection per call site.
Fix: Add a `403` branch that toasts "Not authorized" and/or routes to a denied page.

---

2. Authorization / RBAC (client-side)

> Reminder: this is **UX gating**, not security. It must be mirrored server-side.

2.1 🔴 CRITICAL — Route guards are the only thing separating admin/teacher, and they live in the browser

Evidence: `src/layouts/AdminTeacherLayout.jsx:24-42` decides access by `location.pathname.startsWith("/admin")` vs `role`. The role comes from the JWT/profile, but the *enforcement* is a React `useEffect` that calls `navigate`. Anyone can:

- open devtools and short-circuit the redirect,
- import an admin page bundle directly,
- or simply call the API with their valid teacher token (which, on the backend side, is not role-checked).

Impact: The admin/teacher boundary is not currently enforceable from the frontend alone. This is the same root issue as the domain-separation question — see FRONTEND_DOMAIN_SEPARATION.md.
Fix: Treat client guards as UX only. The boundary must be the backend (`authorize("admin")` / `authorize("teacher")` middleware). Then keep these client guards for clean UX.

2.2 🟠 HIGH — Asymmetric guards: admin routes are permission-gated, teacher routes are not

Evidence:

- `adminRoutes` wraps **every** page via `withPermissionProtection(...)` → `ProtectedRoute` checking `profile.role_access.permissions` against `permissionUtils.js` `SIDEBAR_PERMISSIONS`.
- `teacherRoutes` (`src/router/routes/teacherRoutes.jsx`) wrap **nothing** — no `ProtectedRoute`, no permission map. The only gate is the `role !== "teacher"` check in the shared layout.

Impact: Inconsistent model; teacher pages have a weaker (role-string-only) guard than admin pages. If teachers ever get sub-roles/permissions, there's no structure for it.
Fix: Apply the same `ProtectedRoute` pattern to teacher routes (even with empty `requiredPermissions` for now) so both trees are uniform and future-proof.

2.3 🟡 MEDIUM — `ProtectedRoute` "fails open" / flashes during profile load

Evidence: `ProtectedRoute.jsx:13-20`: `userPermissions = profile?.role_access?.permissions || []`. If `profile` is still loading (it's fetched *after* login in `useAuthStore.js:34`), permissions are `[]` and the `.some(...)` check returns `false` → "Access Denied" flashes for a legitimate admin during the profile-fetch window. Conversely, routes with `requiredPermissions = []` (e.g. dashboard) always pass even with no profile.
Fix: Gate on a `profileLoaded` flag; render a spinner (not "Access Denied") until the profile resolves.

2.4 🔵 LOW — Permission map keyed by string paths is drift-prone

Evidence: `SIDEBAR_PERMISSIONS` in `permissionUtils.js` hardcodes route strings; a comment even notes a `// Fallback just in case` duplicate for `/admin/teacher-qualification(s)`. Renaming a route silently drops its guard.
Fix: Co-locate `requiredPermissions` with the route definition instead of a parallel string map.

---

3. Cross-Site Scripting (XSS)

3.1 🔴 CRITICAL — Unsanitized `dangerouslySetInnerHTML` rendering server/user HTML

Evidence (6 sites):

- `src/components/admin/notification/NotificationModal.jsx:831`
- `src/components/teacher/notification/TeacherNotificationModal.jsx:293`
- `src/layouts/TeacherNotificationDrawer.jsx:172`
- `src/pages/admin/notification/NotificationDetail.jsx:115`
- `src/pages/teacher/notification/NotificationDetail.jsx:109`
- `src/components/admin/programs/ViewComponent.jsx:166`

These render `notification.message` / `messageContent` (rich-text authored elsewhere, via the TipTap editor) directly as HTML. There is **no sanitizer** in the project — `dompurify` / `sanitize-html` are **not** in `package.json`. The one site at `NotificationDetail.jsx:115` only `encodeURI`s `src=` attributes, which does **not** stop `<img onerror=...>`, `<script>`, or `javascript:` payloads.

Impact: Stored XSS. A notification author (or anyone who can influence message HTML) can run JS in every recipient's admin/teacher session — session-riding the very tokens §1.1 protects.
Fix: Add `dompurify` and wrap every payload: `dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(messageContent) }}`. Centralize in one `<SafeHtml>` component so no call site is missed.

3.2 🔵 LOW — RichTextEditor link building

Evidence: `src/components/ui/RichTextEditor.jsx:90-91` prefixes bare URLs with `https://` but doesn't block `javascript:` / `data:` schemes a user could type. Combined with §3.1 (no sanitization on render), a `javascript:` link could execute.
Fix: Allow-list `http(s)` / `mailto` schemes on link insert; rely on §3.1 sanitization on render.

---

4. Configuration, Secrets & Environment Coupling

4.1 🟠 HIGH — API key is shipped in the client bundle

Evidence: `.env` defines `VITE_APP_API_KEY`; `axiosintercepter.js:5,14` sends it as `x-api-key` on every request. **All `VITE_*` variables are inlined into the built JS** and are trivially readable by any user.
Impact: The "API key" provides **zero** security against an end user — it's visible in the bundle and in every network request. If the backend treats this key as an auth/authorization factor, that protection is illusory.
Fix: Treat it as a non-secret routing/identifier at best. Real authorization must come from the JWT + server-side checks. Do not gate anything sensitive on this key.

4.2 🟡 MEDIUM — Hardcoded cross-application URL

Evidence: `src/pages/LoginSelection.jsx:36,42` hardcodes `https://student-iao.xyvin.com/login?programId=...` (a `xyvin.com` dev/staging domain) for the student portal.
Impact: Will break / point at the wrong environment when promoting UAT → prod; not configurable per environment. Relevant to the `osteopathie.eu` rebrand — this URL must become env-driven.
Fix: Move to `VITE_STUDENT_PORTAL_URL` env var.

4.3 🟡 MEDIUM — PDF worker loaded from public CDN (unpkg)

Evidence: `src/pages/teacher/evaluations/ViewSubmission.jsx:267` loads `pdf.worker.min.js` from `https://unpkg.com/...`.
Impact: Runtime dependency on a third-party CDN (availability + supply-chain/integrity risk); also a CSP headache. `pdfjs-dist` is already a local dependency.
Fix: Bundle the worker locally (Vite `?url` import or `pdfjs-dist` worker entry) instead of the CDN.

4.4 🟡 MEDIUM — SPA rewrite config present for Vercel, but deployment target is changing

Evidence: `vercel.json` rewrites all paths to `/` (correct SPA fallback). If you move to nginx for the `osteopathie.eu` rebrand, this fallback must be re-created in nginx (`try_files ... /index.html`). See the domain doc.

---

5. Code Quality & Hygiene

5.1 🔵 LOW — 29 `console.log/error/warn` left in source

Evidence: e.g. `useAuthStore.js:62,72`, plus 27 others across `src`.
Impact: Noisy prod console; some log error objects that may include user/profile data. Minor info leak + noise.
Fix: Strip via a Vite `esbuild.drop: ["console"]` for prod builds, or a logger util gated on `import.meta.env.DEV`.

5.2 🔵 LOW — `useEffect` redirect pattern can double-fire / flash

Evidence: Both `AdminTeacherLayout.jsx` and `Login.jsx:51–59` redirect inside `useEffect`. With React 19 StrictMode (double-invoke in dev) and router transitions this can cause brief flashes/loops. Prefer router `beforeLoad`/loaders for auth gating so redirects happen before render.

5.3 🔵 LOW — Spelling drift in route/folder names

Evidence: `pages/teacher/evaluvations/` (should be "evaluations"; the route is correctly `/teacher/evaluations`, so folder ≠ URL). Harmless but confusing for grep/maintenance.

5.4 🔵 LOW — Tab-state in `localStorage` is fine, but unbounded keys

Evidence: `batchDetailsTab_${id}`, `intakeDetailsTab_${id}` etc. write per-entity keys to `localStorage` that are never cleaned up. Cosmetic; over time accumulates dead keys.

---

6. What was NOT covered (recommended follow-ups)

This pass focused on **auth/session, RBAC, XSS, config/secrets, and hygiene** — the highest-leverage areas and the ones the domain question depends on. Not yet deeply audited:

- **Form validation parity** — are Zod schemas in `src/validations` enforced server-side too? (Client validation is never sufficient.)
- **Bundle size / code-splitting** — 45 admin + 15 teacher routes eagerly imported in `adminRoutes.jsx`/`teacherRoutes.jsx`; no `React.lazy`. Likely a large initial bundle.
- **Accessibility** (a11y) of the custom UI components.
- **i18n completeness** across the 4 locales.
- **React Query cache-key hygiene** (stale data across users — `queryClient.clear()` on logout is good; verify no cross-user leakage on role switch).

Tell me which of these you want and I'll go deep.

---

7. Prioritized Remediation Plan

Priority	Finding	Action	Effort
P0 — now	3.1 XSS	Add `dompurify`; wrap all 6 `dangerouslySetInnerHTML` in a `<SafeHtml>` component	S
P0 — now	2.1 / 4.1	Accept client guards + API key are **not** security; confirm backend enforces RBAC	— (depends on backend)
P1	2.2	Apply `ProtectedRoute` to teacher routes for parity	S
P1	1.2	Single-flight token refresh	S
P1	2.3	Fix `ProtectedRoute` "Access Denied" flash during profile load	S
P2	4.2 / 4.3	Env-var the student portal URL; bundle the PDF worker locally	S
P2	1.4	Global `403` handling in interceptor	S
P3	5.x	Strip console logs in prod; router-loader auth gating; cleanup	M

Legend: S = < 0.5 day, M = ~1–2 days.

Companion document: FRONTEND_DOMAIN_SEPARATION.md — splitting `admin.osteopathie.eu` and `teacher.osteopathie.eu`.